

Terraform + AWS 総まとめ

laC → VPC → EKS → ECR → EBS → ALB → OIDC CI/CD → Destroy

学習の軸

「AWSの部品を知る」だけでなく、Terraformで再現可能な構成にし、KubernetesアプリをAWSへ載せ、CI/CDと権限分離までつなげ、最後に安全に全部消すところまでを1本の流れとして理解する。

実験ベース: ap-northeast-1 / Amazon EKS / 2-node Managed Node Group / frontend + Go backend
+ PostgreSQL

1. 今日やった全体像

ローカルkindで学んだKubernetesの概念を、AWS上のマネージドサービスへ対応付け、Terraformで構築した。最終的にはGitHub

ActionsからOIDCでAWSへ認証し、ECRへpushしてEKSへデプロイできる構成まで到達した。

```
GitHub
↓ OIDC
IAM Role
↓
ECR ← docker buildx / push
↓
EKS
├── frontend Pods
├── backend Pods
├── PostgreSQL StatefulSet
↓ PVC
EBS
```

Internet → ALB → Ingress → Service → Pods

AWS側で使った主要サービス

役割	AWSサービス	Kubernetes / 開発側との対応
ネットワーク	VPC / Subnet / Internet Gateway / Route Table	クラスタが動く土台
Kubernetes	Amazon EKS	kindのクラウド版に近い位置づけ
Worker	EC2 Managed Node Group	kind worker node
コンテナ保存	Amazon ECR	Docker image registry
永続化	Amazon EBS + EBS CSI	PVC / PV の実体
外部公開	ALB + AWS Load Balancer Controller	IngressのAWS実装
認証・認可	IAM / EKS Access Entry / Pod Identity	AWS権限 + Kubernetes権限
IaC state	Amazon S3	Terraform remote state

重要

KubernetesのYAMLは「アプリをどう動かすか」、Terraformは「そのKubernetesが動くAWS基盤をどう作るか」を主に担当する。

2. Terraformの考え方

Terraformは「今AWSがどうなっているか」を手続き的に操作するのではなく、「こういう状態であってほしい」という設定をHCLで記述し、実環境との差分を計算する。

```
HCL (desired state)
↓
terraform plan
↓ 差分確認
terraform apply
↓
AWS resources
↕
terraform state
```

基本コマンド

コマンド	意味	今回の使いどころ
terraform init	provider / module / backend初期化	S3 backend移行、provider取得
terraform fmt	HCL整形	CIのfmt check
terraform validate	構文・設定検証	CIでapply前に検査
terraform plan	変更差分を表示	作成67 / 削除67などを確認
terraform apply	plan内容を反映	VPC/EKS/ECR/IAM作成
terraform destroy	管理リソースを削除	最終片付け
terraform output	output値確認	cluster名、ECR URLなど

今回の理解

TerraformはAWS APIを呼ぶ「自動化スクリプト」以上に、stateを使って「設定と実環境の対応関係を追跡する仕組み」と考えると理解しやすい。

3. Terraformファイル構成

```
terraform/dev/
├─ versions.tf
├─ providers.tf
├─ variables.tf
├─ terraform.tfvars.example
├─ main.tf
├─ outputs.tf
├─ .terraform.lock.hcl
├─ iam/
└─ aws-load-balancer-controller-policy.json
```

責務

ファイル	責務
versions.tf	Terraform / provider version、backend設定
providers.tf	AWS regionやdefault tagsなどprovider設定
variables.tf	外から変更できる入力値の型・default
terraform.tfvars.example	利用者向けの入力例。秘密値は置かない
main.tf	VPC / EKS / ECR / IAMなど主リソース
outputs.tf	cluster名やECR URLなど作成後に使う値
.terraform.lock.hcl	providerバージョン固定。Gitに含める

providerのポイント

```
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project = "aws-kubernetes-lab"
      Environment = "dev"
      ManagedBy = "Terraform"
    }
  }
}
```

profile名をproviderへハードコードするとGitHub Actionsで動かない。ローカルは `AWS_PROFILE=eks-lab` を環境変数として渡し、CIはOIDCで一時認証情報を取得する形にした。

4. StateとS3 Remote Backend

Terraform stateは「HCL上のresource」と「AWS上のresource ID」の対応を記録する重要データ。失うとTerraformが既存リソースを正しく追跡できなくなる。

```
main.tf
resource aws_vpc ...
↓
terraform.tfstate
aws_vpc.this → vpc-xxxx
↓
AWS real resource
```

今回のremote state

```
backend "s3" {
  bucket = ""
  key = "dev/terraform.tfstate"
  region = "ap-northeast-1"
  use_lockfile = true
}
```

- S3 Versioningを有効化してstate履歴を保持。
- Public Access Blockを有効化。stateには機密情報が含まれる可能性があるため公開しない。
- use_lockfile=trueで同時更新を防ぐ。
- local stateから terraform init -migrate-state でS3へ移行した。

重要

S3 backend bucket自体を同じstateで管理すると「stateを消すためのstate」が必要になる問題がある。今回はbootstrapとして手動作成し、destroy後にS3のVersion / Delete Marker / bucket本体を明示的に削除した。

5. AWSネットワーク構成

```
graph TD
    Internet[Internet] --> IGW[Internet Gateway]
    IGW --> VPC[VPC 10.0.0.0/16]
    VPC --> PS1[Public Subnet 10.0.1.0/24 (AZ-a)]
    VPC --> PS2[Public Subnet 10.0.2.0/24 (AZ-c)]
    PS1 --> RT[Route Table]
    PS2 --> RT
    RT --> EKS[EKS Nodes]
```

Internet
↓
Internet Gateway
↓
VPC 10.0.0.0/16
├── Public Subnet 10.0.1.0/24 (AZ-a)
└── Public Subnet 10.0.2.0/24 (AZ-c)
↓
Route Table
0.0.0.0/0 → IGW
↓
EKS Nodes

用語

用語	意味
VPC	AWS上の論理的な専用ネットワーク
Subnet	VPCをAZ単位などで分割したIP範囲
Availability Zone (AZ)	リージョン内の独立したデータセンター群
Internet Gateway (IGW)	VPCとInternetを接続する出口
Route Table	宛先CIDRごとに次の経路を決める
Security Group	ENI/EC2等につく stateful firewall
Network ACL	Subnet単位の stateless firewall

今回の簡略化

学習用途のためpublic subnet中心・NAT Gatewayなし。Productionならprivate subnetへWorkerを置き、NATやVPC Endpoint、ALBのpublic/private分離を検討する。

6. Amazon EKS: kindからAWSへ

Kubernetes

APIやDeployment/Service/StatefulSetなどの概念はそのまま。変わるのは、その裏側のcontrol plane・node・storage・load balancerをAWSがどう実装するか。

kind	EKS
kind control-plane container	AWS managed EKS control plane
kind worker container	EC2 Managed Node Group / Fargate
local-path StorageClass	EBS / EFS CSI Driver
Ingress Controller	AWS Load Balancer Controller
Docker local image	Amazon ECR
ローカル認証	IAM + EKS Access Entry

Managed Node Group

```
EKS Control Plane
↓ scheduler
Managed Node Group
├── EC2 t3.small
└── EC2 t3.small
↓ kubelet
Pods
```

今回のNode Groupは min=1 / desired=2 / max=2。Terraform moduleではdesired_sizeが作成後に無視される設計があり、実際のdesired数をCLIで2へ変更した場面もあった。

EKS Add-ons

- vpc-cni: Pod networking
- kube-proxy: Service通信
- coredns: cluster DNS
- eks-pod-identity-agent: Pod Identity
- aws-ebs-csi-driver: EBS volume連携

7. ECRとコンテナイメージ

ECRはAWSのcontainer registry。frontend / backendを別Repositoryにし、GitHub Actionsからcommit SHAタグでpushした。

```
source code
↓ docker buildx
linux/amd64 image
↓ push
Amazon ECR
↓ image pull
EKS Node
↓
Pod
```

Mac arm64で起きる注意点

Apple Siliconで普通にbuildするとarm64 imageになる。一方、今回のEKS nodeはAL2023 x86_64だったため、buildxでplatformを明示した。

```
docker buildx build \
--platform linux/amd64 \
--push ...
```

障害パターン

Podが ErrImagePull / ImagePullBackOff の場合は、image名・tag・ECR認証・CPU architectureを確認。今回もlocal image指定をAWS用ECR imageへ直して解決した。

8. PostgreSQL + EBS CSI

```
PostgreSQL StatefulSet
↓
PVC (1Gi, RWO)
↓
StorageClass gp3
↓
EBS CSI Driver
↓
Amazon EBS volume
```

Kubernetes側はPVCを要求するだけで、AWS EBS CSI DriverがAWS APIを使ってEBSを作り、PVとして結び付ける。

今回のポイント

- EBS CSI DriverへPod IdentityでAmazonEBSCSIDriverPolicyV2を付与。
- PVCがBoundになるとEBS volumeが作成される。
- Podを消してもPVC/PV/EBSが残ればデータは保持される。
- PostgreSQLではEBSのlost+found問題を避けるため PGDATA=/var/lib/postgresql/data/pgdata を利用。
- init.sqlは /docker-entrypoint-initdb.d に置くと「新規DB初期化時のみ」実行される。

削除時

PVCを先に消そうとしてもPodが使用中なら pvc-protection でTerminatingになる。StatefulSet/Podを止めてVolumeを解放するとPVC→PV→EBSの順に削除できる。

9. ALB / Ingress / AWS Load Balancer Controller

```
Browser
↓
AWS Application Load Balancer (ALB)
↓
Kubernetes Ingress
↓
frontend Service
↓
frontend Pods
↓ HTTP
backend Service
↓
backend Pods
```

Ingress YAMLそのものがALBではない。AWS Load Balancer ControllerがIngressを監視し、AWS APIを呼んでALB・Target Groupなどを作る。

Controllerの権限

```
aws-load-balancer-controller Pod
↓ ServiceAccount
EKS Pod Identity Association
↓
IAM Role
↓
ELBv2 / EC2 API
```

Helmで `serviceAccount.create=false` にしたのにServiceAccountが存在せずPod作成失敗になった。手動でSAを作成して復旧したが、再現性の観点ではHelm/Terraformで管理すべき改善点として残した。

削除順序が重要

```
Ingress削除
↓ ALB削除を確認
PVC/EBS削除
↓
Load Balancer Controller uninstall
↓
terraform destroy
```

なぜ？

先にEKSやControllerを消すと、Ingressが作ったALBなど「Kubernetes外部のAWSリソース」をControllerが後片付けできなくなる可能性がある。

10. GitHub Actions CI/CD + OIDC

```
git push (main)
↓
GitHub Actions
↓ OIDC token
AWS STS AssumeRoleWithWebIdentity
↓
IAM Role
├── ECR push
└── EKS deploy
↓
kubectl set image / rollout
```

OIDCを使う理由

GitHub Secretsへ長期Access Keyを保存せず、workflow実行時だけ短命なAWS credentialを取得できる。漏洩リスクとcredential rotation負担を減らせる。

Roleを分離

Role	権限
Deploy Role	ECR push + EKS deployに必要な権限
Terraform Plan Role	ReadOnlyAccess + tfstate S3アクセス

Terraform planにDeploy Roleを流用せず、用途ごとにRoleを分けた。これはLeast Privilegeと責務分離の実践。

Path filter

READMEやTerraformだけのpushでアプリdeployが走らないよう、frontend/backend関連pathとdeploy workflowだけをtrigger対象にした。

11. IAM / Pod Identity / RBAC

AWSでは「AWS APIを呼べる権限」と「Kubernetes APIを操作できる権限」を分けて考える。

GitHub → IAM Role → AWS APIs

└→ EKS Access Entry → Kubernetes access

Pod → ServiceAccount → Pod Identity → IAM Role → AWS APIs

Kubernetes user/SA → RoleBinding → Role → K8s API permissions

今回のセキュリティ原則

- 長期Access KeyをGitHubへ置かずOIDC。
- Deploy RoleとTerraform Plan Roleを分離。
- Podには必要なAWS権限だけをPod Identityで付与。
- Secretの実値はGitへcommitせず、exampleだけ公開。
- S3 stateをprivate + versioning。
- Node metadataはIMDSv2 required。

改善点

初期セットアップではroot profileを使ったが、継続運用ではrootを日常利用せず、管理用IAM Identity Center / IAM Roleへ移すのが望ましい。

12. 今日踏んだ障害と診断

症状	原因	学び
Node NotReady	VPC CNI等add-on不足	EKS control planeだけではPod networkは完成しない
EBS CSI 409	途中apply後のPod Identity association残存	AWS側実体とTerraform stateのズレを疑う
ErrImagePull	local image / ECR tag不一致	describe + image名確認
Postgres初期化失敗	lost+found	PGDATAをsubdirectoryへ
LBC Pod FailedCreate	ServiceAccount未作成	Helm valuesと実際の依存resourceを確認
GitHub OIDC失敗	subject形式不一致	OIDC trust conditionを実際のsubへ合わせる
Terraform CI認証失敗	providerにlocal profile固定	providerを環境credential対応へ
Terraform plan AccessDenied	plan Roleのread不足	Roleを分離し必要なread/state権限付与

AWS/Terraform障害診断の順序

1. terraform plan / error本文
2. どのresourceで失敗したか
3. AWS CLIで実体確認
4. IAM権限 / trust policy
5. Terraform stateとの対応確認
6. dependency / deletion order
7. 必要なら手動修復 → 再plan

13. Destroyと「本当に消えたか」の確認

IaCは作るだけでなく、壊して再作成できることが価値。今回は課金を止めるため、外部リソースを先に片付けてからTerraform管理分を67 resources destroyした。

1. Ingress delete → ALB = [] を確認
2. StatefulSet停止 → PVC/PV削除 → PostgreSQL EBS消失
3. AWS Load Balancer Controller uninstall
4. ECR imageを全削除 (untaggedも含む)
5. terraform destroy
6. terraform plan → 67 to add / 0 destroy = stateが空
7. EKS=[] / ALB=[] / EC2=terminated を確認
8. S3 state versions + delete markers削除
9. S3 bucket delete → head-bucket 404

ECRの罠

Repositoryにimageが残っているとECR Repository削除が失敗することがある。tagged imageを消した後もbuildx由来のuntagged digestが残ったため、list-imagesで再確認して全削除した。

KMS

KMS Keyはdestroy時に即時消滅ではなく「削除予定」になることがある。Terraform上から消えてもAWS Consoleに一定期間表示されるのは仕様。

最終確認の思想

「terraform destroyが成功した」だけで終わらず、課金リスクの大きいEKS / ALB / EC2 / EBS / ECRをAWS APIでも確認する。

14. コスト観点

学習環境では「使わない時間に止める」だけでなく、「どのresourceが時間課金・容量課金なのか」を把握することが重要。

resource	コスト感	今回の対応
EKS Control Plane	クラスタ稼働時間で課金	学習終了後destroy
EC2 Node	インスタンス稼働時間	Node Group destroy
ALB	稼働時間 + LCU	Ingress削除で先にALB削除
EBS	容量ベース	PVC/PV削除を確認
ECR	保存容量	imageとrepoを削除
S3 tfstate	小容量なら非常に小さい	今回はbucketまで削除
KMS customer key	key利用/保有	destroyで削除予定

Productionなら

毎回destroyするのではなく、dev/stg/prodを分離し、devだけ夜間停止・Node scale down・spot・autoscaling・TTL環境などを組み合わせる。

15. よく使ったコマンド Cheat Sheet

Terraform

```
AWS_PROFILE=eks-lab terraform init
AWS_PROFILE=eks-lab terraform plan
AWS_PROFILE=eks-lab terraform apply
AWS_PROFILE=eks-lab terraform destroy
terraform fmt -check
terraform validate
terraform output
```

EKS / EC2 / ELB / ECR

```
aws eks list-clusters --region ap-northeast-1 --profile eks-lab
aws eks update-kubeconfig --name --region ap-northeast-1 --profile eks-lab
aws elbv2 describe-load-balancers --region ap-northeast-1 --profile eks-lab
aws ec2 describe-volumes --region ap-northeast-1 --profile eks-lab
aws ecr list-images --repository-name --region ap-northeast-1 --profile eks-lab
aws ecr batch-delete-image --repository-name --image-ids file://images.json ...
```

S3 state削除

```
aws s3api list-object-versions --bucket --profile eks-lab
aws s3api delete-objects --bucket --delete file://versions.json --profile eks-lab
aws s3api delete-bucket --bucket --region ap-northeast-1 --profile eks-lab
aws s3api head-bucket --bucket --profile eks-lab
```

16. 用語辞典

用語	一言で
IaC	Infrastructure as Code。インフラをコードで再現する考え方
HCL	Terraform設定を書く言語
Provider	TerraformとAWS等のAPIをつなぐplugin
Module	Terraform resource群の再利用単位
State	Terraform resourceと実resourceの対応表
Backend	stateの保存先・locking方式
EKS	AWS managed Kubernetes
Managed Node Group	EKS worker EC2群をAWSが管理する仕組み
ECR	AWS container image registry
CSI	Kubernetesとstorage providerを接続するinterface
EBS	EC2向けblock storage
ALB	HTTP/HTTPS向けLayer 7 Load Balancer
OIDC	外部IdPのtokenを使った認証連携
STS	一時credentialを発行するAWS service
IAM Role	一時的に引き受けるAWS権限のまとまり
Pod Identity	EKS PodへIAM Roleを割り当てる仕組み
Access Entry	IAM principalにEKS cluster accessを付与する仕組み
Least Privilege	必要最小限の権限だけ付与する原則

最後の整理

Kubernetesで「アプリのdesired state」を管理し、Terraformで「AWS infrastructureのdesired state」を管理する。GitHub Actionsはその変更を自動で検証・反映し、IAM/OIDCが「誰が何をできるか」を制御する。この4層を分けて見ると全体がかなり整理しやすい。